

Predict Once, Query Many: World Models that Forecast Conditional Trajectory Abstractions

Anonymous CVPR submission

Paper ID *****

Abstract

Visual world models predict action-conditioned changes in observations or latent features. Their prediction target determines what the dynamics model must learn and what information is available to a planner. We investigate a target learned for a complete action segment: Conditional Trajectory Abstraction (CTA). A training-only encoder maps observed context and sampled future observations to 16 quantized tokens. A goal-conditioned reader and a future-feature decoder jointly supervise this target. The world model then learns to forecast its token distributions from context and a proposed action chunk, using categorical prediction and reader-score consistency. At deployment, the reader evaluates the predicted coordinate means without receiving candidate actions; a forecast can be reused across goal queries. This yields a 48-scalar segment interface while retaining a separately learned model of action consequences. On identical eight-candidate PushT development banks, CTA recovers 0.680 of the available geometry improvement, versus 0.635 for endpoint prediction and 0.502 for a parameter-matched direct scorer. In 200 development episodes it achieves 70.0% success, compared with 61.0% for the proposal policy and 69.0% for endpoint prediction. Component benchmarks characterize scaling with candidates and goal views. The results support learned segment targets for visual world modeling; independent multi-seed control and semantic-goal transfer remain to be established.

1. Introduction

Visual world models learn how an environment changes under an agent's actions, enabling a planner to evaluate predicted outcomes before execution [3, 8, 21]. A central design choice is the *prediction target*. Pixel reconstruction, spatial visual features, compact tokens, and reward-oriented latent states preserve different information and impose different computational demands. Understanding this choice matters

both for learning dynamics and for using those dynamics in decisions.

Recent visual world models build on strong pretrained encoders. DINO-WM forecasts DINOv2 patch features [17, 21], and V-JEPA 2-AC forecasts features of a pretrained video encoder [3]. Such representations provide broad visual supervision and support image-goal planning. CompACT reduces the spatial representation to a small set of discrete observation tokens [11]. These advances leave another design dimension open: the temporal unit and supervision of the prediction target. Instead of forecasting a sequence of independently represented observations, can a world model forecast one learned description of an action segment, conditioned on the context already available to its readout?

We investigate this question through *Conditional Trajectory Abstraction* (CTA). During training, a target encoder observes context C and the actual future segment τ of an action chunk, then produces a compact code S . A goal-conditioned reader learns to evaluate (C, S, g) , while a feature decoder reconstructs sampled future features from (C, S) . The world model $p_\theta(S \mid C, \mathbf{A})$ forecasts this target from context and proposed actions. Thus the predictive object is learned from action consequences rather than fixed to the visual encoder's output. The reader never receives $\mathbf{A}$: candidate-specific information enters through the world model's forecast. Actual future observations are used only in training.

Conditioning the source and readout on the same observed context follows the idea of contextual coding [13, 15]. Representing a multi-step segment also connects to compact trajectory models [10]. CTA brings these ideas to action-conditioned visual prediction, with a target shaped by both future-feature supervision and a learned decision readout. This differs from searching for actions in a trajectory-code space, and from compressing each observation before an autoregressive rollout. It also differs from direct value guidance [16]: direct scoring is a valid alternative that need not predict an explicit future target. Whether future supervision helps is a comparison to measure, rather than an assumption of the method.

CTA predicts all segment-code coordinates in parallel.

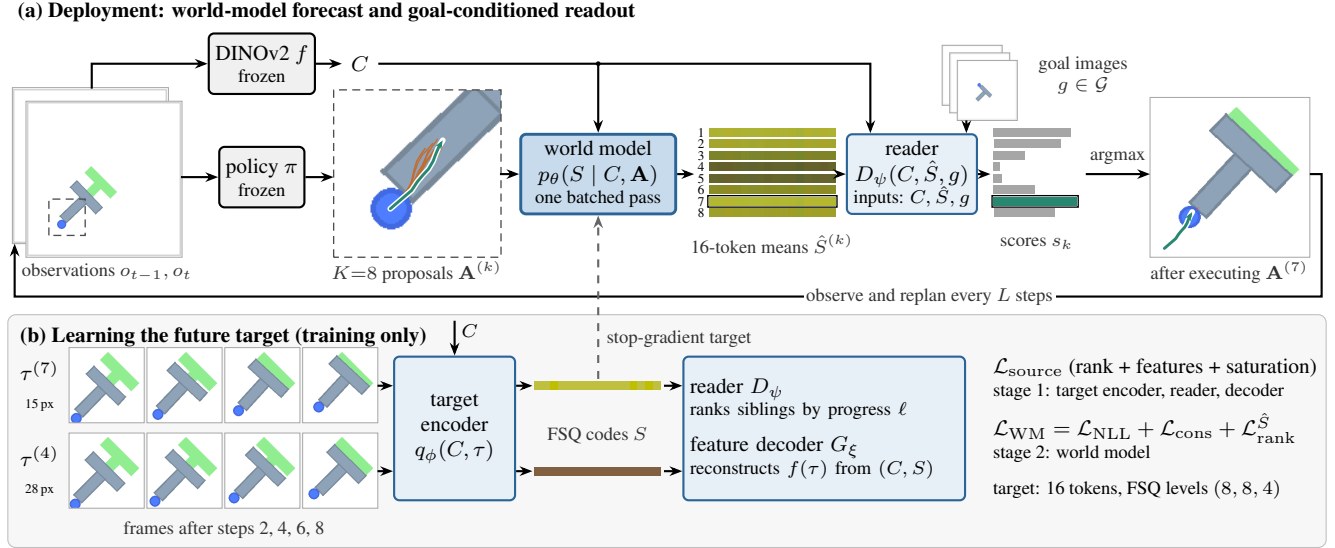


Figure 1. **A world model with a learned conditional prediction target.** (a) Frozen visual features and proprioception form C ; a diffusion policy supplies eight action chunks. The world model forecasts one 16-token segment representation per candidate. Each row of colored cells shows a forecast mean, with three coordinates encoded as RGB. The reader evaluates $(C, \hat{S}, g)$; bars show relative scores normalized within this example. Green marks the selected proposal, which is executed before replanning. The goal-independent forecasts can be cached across reader queries. (b) During target learning, sibling proposals are executed from cloned states. Sampled actual futures enter $q_\phi(C, \tau)$, supervised by ranking and future-feature reconstruction. Frames, paths, codes, and scores come from one archived PushT checkpoint/decision and illustrate the interface; they are not aggregate performance evidence. Images are cropped simulator renders.

The code encoder and world model receive no goal input, so predicted codes can be cached while a reader evaluates different goal queries: *predict once, query many*. This input separation is shared with other outcome-based planning systems [5, 18]; our focus is the learned conditional segment target within that separation. The present PushT experiments use multiple image views of one target pose and do not evaluate transfer to unseen semantic goals.

We evaluate world-model targets through a common proposal-based planner. A frozen diffusion policy proposes candidate chunks, and each learned method selects among them. Offline comparisons hold states and complete action banks fixed; closed-loop comparisons measure the consequences of repeated selection. On eight-candidate development banks, CTA improves geometry ranking over endpoint prediction and both ordinary and parameter-matched direct scorers. In closed-loop development it improves success over the proposal policy, while learned alternatives remain close and their differences uncertain. Timing measurements expose how predictor/readout cost scales with candidate count and repeated queries.

Our contributions are:

- A conditional segment representation for visual world modeling, learned from observed futures and shared context rather than prescribed as a sequence of frame features.
- A world-model training architecture combining a quan-

tized future target, a goal-conditioned reader, auxiliary feature reconstruction, and forecast-to-reader consistency. Its deployment readout has no access to proposed actions or observed futures.

- A comparison of learned prediction targets under shared proposal banks, including a capacity-matched direct control, closed-loop development results, and candidate/query scaling measurements.

2. Related Work

Visual and latent world models. Latent dynamics models such as PlaNet learn compact state transitions from images for planning [8]. Joint-embedding predictive architectures predict representation-space targets instead of generating pixels [2]. DINO-WM learns action-conditioned prediction on frozen DINOv2 features [21]; V-JEPA 2-AC post-trains an action-conditioned predictor on a frozen video representation [3]. Their spatial targets support rich visual prediction, but do not directly answer how a single segment-level target should be learned. CTA retains a pretrained visual input representation while learning an additional conditional target for an action segment.

Compact and conditional prediction targets. CompACT learns compact discrete image representations for latent

world-model rollouts [11]. Delta-IRIS introduces context-aware temporal tokenization [15], and contextual video compression uses shared side information when coding observations [13]. TAP encodes state-conditioned trajectories in a discrete latent action space and searches over those codes [10]. CTA instead forecasts a future code induced by an externally supplied action chunk, then evaluates it with a learned reader. Compactness, conditioning, and trajectory coding have clear precedents; the distinction is the source target and forecast/readout interface. A matched compact per-frame control would further isolate the temporal choice.

World models for policy steering. Generative Predictive Control ranks or refines policy proposals using predicted consequences [18]. FOREWARN aligns latent prediction with visual-language reasoning [20]; DreamSteer uses predicted visual progress to steer deployed policies [5]. Robot Critics learns progress comparisons from success/failure rollouts and applies them to generated consequences [19]. DynaGuide guides diffusion denoising with learned dynamics [6]. CTA uses proposal selection to study its world-model target; proposal selection and action-blind outcome evaluation are established planning interfaces. Our endpoint comparator is a matched learned model, not a reproduction of GPC.

Decision-oriented representations and direct guidance. Value equivalence and value-equivalent sampling formalize preserving model information needed for value-based decisions [1, 7]; TD-MPC2 learns control-oriented latent dynamics [9]. V-GPS and RoboMonkey evaluate proposed actions directly [12, 16]. These approaches motivate decision-aware representation learning and provide useful alternatives to explicit prediction. CTA supervises the prediction target using a ranking reader and future features. We compare with direct scoring without assuming that an explicit world model is necessary for good control.

3. World-Model Prediction Targets

Let C_t contain recent visual observations and proprioception. A candidate action chunk $\mathbf{A}_t = (a_t, \dots, a_{t+L-1})$ induces a future segment τ_t , comprising sampled future image features and endpoint proprioception. The world model forecasts a representation of τ_t before any of those observations is available. A goal query g specifies how the prediction is evaluated.

CTA learns the target jointly from context and future:

$$S = q_\phi(C, \tau), \quad p_\theta(S | C, \mathbf{A}), \quad s = D_\psi(C, \hat{S}, g), \quad (1)$$

where $\hat{S} = \mathbb{E}_{p_\theta}[S]$ in the implemented planner. The source encoder q_ϕ operates only during training. At deployment, the reader receives context, forecast, and goal, with no candidate

Architecture		Forecast object	Evaluation	
Spatial WM	latent	Sequence of spatial feature grids	Feature reader	cost /
Endpoint WM		Endpoint features + proprioception	Learned reader	future
Direct scorer		No explicit future target	$D(C, \mathbf{A}, g)$	
CTA WM		One conditional segment code	$D(C, \hat{S}, g)$	

Table 1. Prediction targets compared in this study. Direct scoring is a separate alternative, rather than a special case of the finite future-code encoder.

actions. Therefore the world model is the only candidate-specific input path to the reader.

The target consists of $M = 16$ finite-scalar-quantization tokens with levels (8, 8, 4) [14]. Each token has 256 grid values. The source alphabet bounds target capacity, while conditioning allows the encoder and readout to share context. Our training objective shapes what the code retains through ranking and feature reconstruction; it does not explicitly minimize conditional entropy. Deployment uses a continuous 16×3 coordinate mean. Source token capacity and deployed numerical precision are distinct quantities.

For control, a frozen policy supplies K proposals. The reader scores each predicted segment, and the planner executes the highest-scoring chunk, using the smallest index to break ties, before replanning. A label $\ell(C, \tau, g)$ supplies supervision on training branches; it is not a deployment input. Offline evaluation uses identical banks, whereas closed-loop states and subsequent banks depend on earlier choices.

4. CTA World Model

CTA separates learning a useful prediction target from forecasting it. First, a conditional target encoder, reader, and feature decoder learn from observed action consequences. Second, a world model learns to forecast the frozen target from context and actions. At deployment, only the world model and reader are used (Fig. 2).

4.1. Learning the conditional future target

Context and future features. We encode images with frozen DINOv2 ViT-S/14 [17] and project features to 128 dimensions using PCA fitted on training images. Native 96×96 observations are resized to 224×224 . Context C contains 256 current-image tokens, 64 pooled previous-image tokens, and one proprioception token embedding current and previous agent positions. The source future τ contains an endpoint feature grid, three pooled intermediate grids, and endpoint proprioception. For $L = 8$, intermediate samples are steps (2, 4, 6); for $L = 15$, they are (4, 8, 11). Samples beyond termination repeat the final observation.

Conditional encoding. Sixteen learned queries cross-attend to the context and future tokens through a three-layer Transformer decoder. A projection produces three coordinates per query; finite scalar quantization (FSQ) [14] maps them to levels $(8, 8, 4)$ with straight-through gradients. For level l , the grid is $\mathcal{Q}_l = \{(r - \lfloor l/2 \rfloor) / \lfloor l/2 \rfloor\}_{r=0}^{l-1}$. The resulting segment code has 16×3 coordinates, independent of the number of sampled future feature tokens. It represents the future jointly with the observed context rather than reconstructing a standalone trajectory.

Decision and feature supervision. A goal-conditioned Transformer reader produces $D_\psi(C, S, g)$. Training banks contain K sibling futures executed from the same initial state. Let ℓ_{bi} denote the training utility of candidate i in bank b , and let $\mathcal{P} = \{(b, i, j) : \ell_{bi} - \ell_{bj} > \epsilon\}$. We train the reader and encoder using

$$\mathcal{L}_{\text{rank}}(s, \ell) = \frac{1}{|\mathcal{P}|} \sum_{(b, i, j) \in \mathcal{P}} \text{softplus}[-(s_{bi} - s_{bj})]. \quad (2)$$

An empty pair set contributes zero. Within-bank supervision reflects the planner’s selection operation without requiring globally comparable scores.

A separate decoder $G_\xi(C, S)$ reconstructs endpoint and intermediate *image* features as residuals from current features. The endpoint and intermediate MSE terms are normalized by their train-set copy-current errors and averaged as $\mathcal{L}_{\text{feat}}$. This provides visual supervision beyond the scalar ranking label. The source-stage objective is

$$\mathcal{L}_{\text{source}} = \mathcal{L}_{\text{rank}} + \alpha \mathcal{L}_{\text{feat}} + \lambda_{\text{sat}} \mathbb{E}[(|z_{\text{pre}}| - 1.5)_+^2]. \quad (3)$$

The final term regularizes pre-quantization saturation. Both the reader and feature decoder share context with the encoder; neither receives candidate actions.

4.2. Forecasting the segment with a world model

The predictor encodes C and the candidate action sequence. Each action a_h is embedded as $[a_h/512 - 1/2, (a_h - p_t)/64]$, combining absolute and relative positions. Sixteen queries decode a segment forecast in parallel through four decoder layers, following a four-layer context/action encoder. There is no autoregressive rollout of intermediate feature grids.

The output is a categorical distribution for each token coordinate:

$$p_\theta(S | C, \mathbf{A}) = \prod_{m=1}^M \prod_{j=1}^3 p_{\theta, m, j}(S_{m, j} | C, \mathbf{A}), \quad (4)$$

$$\hat{S}_{m, j} = \sum_{r=0}^{l_j-1} p_{\theta, m, j}(r | C, \mathbf{A}) \mathcal{Q}_{l_j}[r], \quad (5)$$

where $(l_1, l_2, l_3) = (8, 8, 4)$. The shared Transformer models interactions in its hidden computation, while the categorical output distribution factorizes over coordinates. We train with the mean coordinate cross-entropy $\mathcal{L}_{\text{NLL}}$ against the realized source code. The reader evaluates the continuous coordinate mean $\hat{S}$; this is a deterministic readout of the forecast distribution, not an expectation of the nonlinear reader’s score.

4.3. Aligning forecasts with the decision readout

During world-model training, the source encoder and reader are frozen. Let $u_k = D_\psi(C, S^{(k)}, g)$ be the teacher scores and $s_k = D_\psi(C, \hat{S}^{(k)}, g)$ the predicted scores. We align their relative values within a bank:

$$\mathcal{L}_{\text{cons}} = \text{SmoothL1}\left(\frac{s - \bar{s}}{T}, \frac{\text{sg}(u) - \bar{u}}{T}\right). \quad (6)$$

Here T is a train-only RMS teacher-score scale, floored at 10^{-3} . Centering removes a bank-wide offset, which does not affect action selection. Weighted ranking additionally multiplies each term in Eq. (2) by $w_{ij} = \min(1, (\ell_i - \ell_j)/\sigma)$, with the same pair-count denominator. The complete world-model objective is

$$\mathcal{L}_{\text{WM}} = \mathcal{L}_{\text{NLL}} + \mathcal{L}_{\text{cons}} + \mathcal{L}_{\text{rank}}^w. \quad (7)$$

Gradients through the frozen reader guide the forecast toward distinctions used by the planner; the reader’s parameters remain unchanged.

For a fixed bank, a maximum centered score error ϵ implies source-reader regret at most 2ϵ . A unique source-score margin larger than 2ϵ therefore preserves the selected action. The supplement proves this elementary stability bound. It motivates readout consistency but does not establish native task regret without additional calibration assumptions.

4.4. Training recipe and planning

Training executes standard and perturbed policy proposals from cloned simulator states. All sibling branches from an episode remain in one split. The dense training utility is the negative mean distance between eight corresponding block and target vertices, divided by 512. These privileged labels supervise training; deployment uses images, agent positions, actions, and image goals. Native evaluation uses PushT coverage success rather than a threshold on the training label.

We use AdamW with learning rate 3×10^{-4} , weight decay .05, dropout .1, and 16 banks per update. Source and prediction stages run for 16,000 and 10,000 updates, respectively, with development checkpoint selection. We set $\alpha = \lambda_{\text{sat}} = 1$, $\epsilon = .001$, and $\sigma = .01$. All Transformer modules use width 256 and eight heads; the reader has four layers and the reconstruction decoder two. The

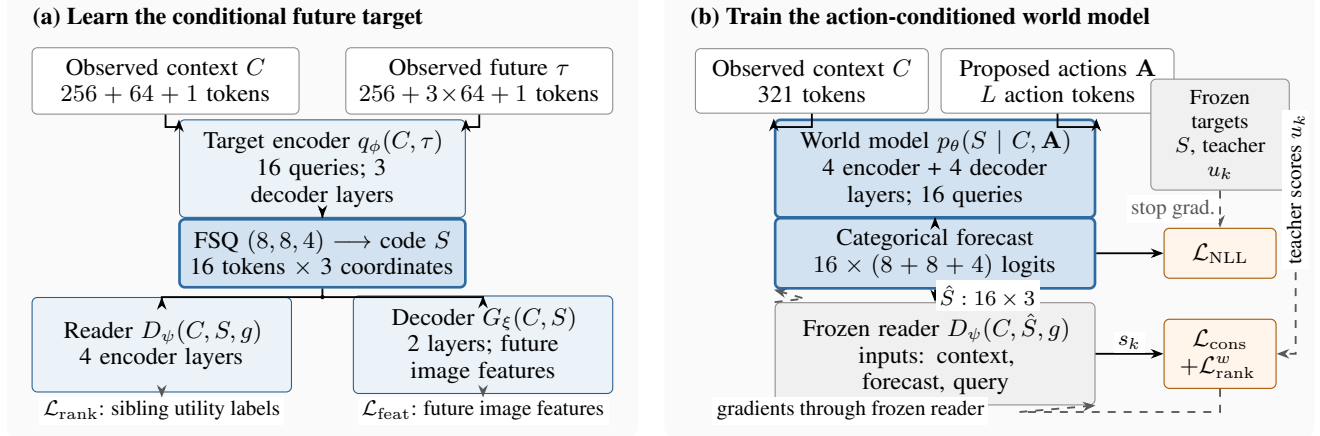


Figure 2. **Training the prediction target and the world model.** Stage 1 learns a 16-token conditional code from observed context and sampled future features, using sibling ranking and feature reconstruction. Stage 2 freezes the source encoder and reader, forecasts coordinate distributions from (C, A) , and optimizes both code likelihood and the reader’s relative scores. Dashed arrows denote supervision or gradient flow through a frozen module. At deployment, the source path is removed and the continuous 16×3 forecast is the only candidate-specific input to the goal-conditioned reader. All Transformer modules use width 256 and eight attention heads.

Algorithm 1 CTA world-model planning at one decision

Require: observed context C , goal features $\mathcal{G}$, policy π

- 1: $\{A^{(k)}\}_{k=1}^K \leftarrow \pi(C)$
- 2: $\hat{S}^{(k)} \leftarrow \mathbb{E}_{p_\theta(S|C, A^{(k)})}[S]$ for all k
- 3: $s_k \leftarrow |\mathcal{G}|^{-1} \sum_{g \in \mathcal{G}} D_\psi(C, \hat{S}^{(k)}, g)$
- 4: choose the smallest index k^* maximizing s_k
- 5: execute up to L actions; observe and replan

supplement specifies data mixtures, selection views, selected checkpoints, and optimizer grouping.

The prediction is independent of g , so context features and segment forecasts can be cached across reader queries. A new raw goal image requires its own feature extraction. This separation is an architectural property; the experiments measure its component cost under repeated queries.

5. Experiments

We study three questions: how the learned world-model target affects ranking on identical proposals, whether repeated selection improves closed-loop control, and how prediction/readout computation scales with candidates and queries. All main comparisons use deployable learned methods.

5.1. Evaluation protocol

Task and planner. PushT requires moving a planar T-shaped block into an image-specified pose. A frozen Diffusion Policy [4] supplies $K = 8$ action proposals per decision. We evaluate execution horizons $L = 8$ and $L = 15$, with a 300-step episode limit. Native success requires post-action coverage greater than .95 before the limit. For ranking, each

method scores the same complete candidate banks. For control, each method replans after executing its chosen chunk; visited states then depend on earlier selections.

Baselines. P0 executes the first policy proposal. END is an endpoint world model: it predicts endpoint image features and proprioception, then uses a learned reader; its objective combines normalized feature MSE, score consistency, and weighted ranking. DIR directly scores (C, A, g) using the same dense labels. DIR-large is a 14-layer, 11.31M-parameter direct scorer, compared with CTA’s 10.95M, END’s 11.14M, and DIR’s 6.57M. These counts exclude the shared frozen image encoder and policy. The original CTA/DIR source groups have distinct weights but share an optimizer and global gradient clipping; DIR-large is trained independently. Full optimization and selection details are in the supplement.

The official DINO-WM checkpoint [21] provides a spatial-feature world-model reference on the same proposals. It retains its own image encoding, velocity inputs, training, and latent-distance cost; our arms use PCA features and agent-position history. This external comparison therefore matches candidate banks and horizons, while the CTA/END/DIR comparisons more closely match observation and training interfaces. DIR-large is evaluated on fixed banks; a completed closed-loop run is not available for this arm.

Evaluation population and uncertainty. The principal ranking population comprises 5,558 logged P0 decisions from 200 development initializations (roots 2200–2399), of which 3,433 have nonconstant geometry labels. Training and

Paired RG difference	Estimate	95% CI
CTA – END	+0.045	[.011,.079]
CTA – DIR	+0.197	[.153,.241]
CTA – DIR-large	+0.178	[.138,.220]
CTA – DINO-WM	+0.346	[.294,.398]

Table 2. Paired comparisons using the same root resamples. The external DINO-WM comparison retains the input/training differences in Sec. 5.1.

checkpoint-selection banks are separate, but these evaluation roots have been reused during development. Each main model has one training seed. We report 95% root-cluster bootstrap intervals and paired contrasts, rather than treating decisions as independent samples. Historical control uses the old batched proposal sampler, whose batch-group sensitivity is documented in the supplement. Independent multi-seed control under the corrected sampling protocol remains necessary for a submission-strength benchmark.

Ranking metric. For bank b , utility y_{bk} , and selected candidate $\hat{k}_b$, geometry retained gap is

$$\text{RG} = \frac{\sum_b (y_{b\hat{k}_b} - y_{b0})}{\sum_b (\max_k y_{bk} - y_{b0})}. \quad (8)$$

It measures aggregate improvement over candidate zero relative to the available bank improvement. It is not success probability or retained information; it can be negative and is undefined at zero denominator. Ties use the smallest candidate index. Bootstrap estimates aggregate numerator and denominator after resampling roots and use identical resamples for paired comparisons.

5.2. World-model targets on fixed proposals

Figure 3 compares deployable methods on identical $L = 8$ banks. CTA reaches RG .680, versus .635 for endpoint prediction and .483 for direct scoring. The paired CTA–END difference is +0.045 [.011,.079], and CTA–DIR is +0.197 [.153,.241] (Tab. 2). Thus the conditional segment target improves this development ranking measure relative to the tested endpoint and direct alternatives.

Capacity control. Increasing the direct scorer to 11.31M parameters yields RG .502 [.445,.553], still below CTA by +0.178 [.138,.220]. The direct model’s own improvement is +0.019 [−.021,.058]. The observed CTA advantage is therefore not explained by this parameter increase. The control does not isolate conditional encoding, future supervision, temporal sampling, or factorization individually; those require separate matched ablations. In particular, a goal-independent direct feature extractor followed by a reader

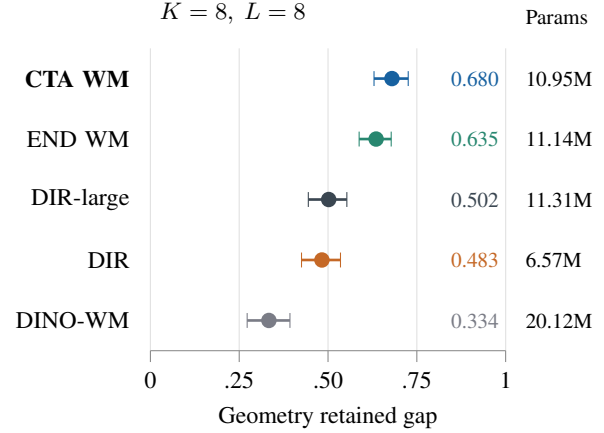


Figure 3. **Learned methods on identical proposals.** $L = 8$, $K = 8$, 5,558 decisions from 200 development roots. Points and horizontal bars show geometry retained gap and 95% root-cluster bootstrap intervals; parameter counts exclude the frozen encoder and policy. DINO-WM uses the external checkpoint and input interface described in Sec. 5.1. All plotted methods score candidates without observing their future.

Method	$L = 8$		$L = 15$	
	Success (%)	Score	Success (%)	Score
P0 policy	61.0	.961	51.5	.911
DIR	67.5	.973	51.5	.920
END WM	69.0	.956	54.0	.936
DINO-WM	68.0	.974	56.0	.932
CTA WM	70.0	.960	60.5	.930

Table 3. **Historical closed-loop development results.** Each column uses 200 roots, one training seed, and the old batched proposal protocol. Score is maximum post-action coverage divided by .95 and clipped to one. Horizon-specific models use different training mixtures. Paired uncertainty is reported in the text and supplement; this table is not a sealed or canonical evaluation.

would test how much of the gain comes from the prediction target rather than the two-stage interface.

5.3. Closed-loop development control

Table 3 reports success and normalized maximum coverage for 200 common initial roots. CTA achieves 70.0% success at $L = 8$ and 60.5% at $L = 15$, improving over the proposal policy by nine percentage points at both horizons. Paired 95% intervals for CTA–P0 are [1,17.5] and [1,17] points, respectively. These results show that the learned forecast can guide repeated action selection in this development setting.

The differences between learned methods are less conclusive. At $L = 8$, CTA–END is +1 point [−7.5, 9.5], CTA–DIR +2.5 [−5.5, 10.5], and CTA–DINO-WM +2 [−6, 10]. At $L = 15$, these differences are +6.5 [−2.5, 15.5], +9 [5,17.5], and +4.5 [−4.5, 13.5]. The exact McNemar test

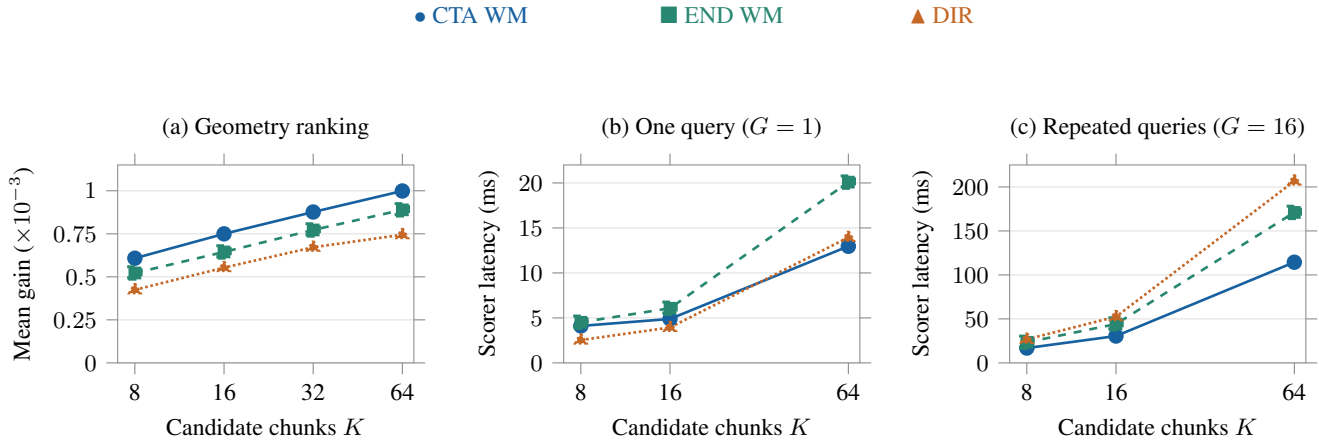


Figure 4. **Quality and component cost under larger candidate banks.** (a) Mean geometry improvement over candidate zero on 2,810 logged P0 decisions from 100 development roots, $L = 8$; paired uncertainty is reported in the text. (b,c) Median synchronized prediction/readout latency on ten initial states, H100 3g.40gb MIG. All points are archived measurements; lines connect measured candidate counts. Timing excludes shared encoding, policy sampling, and simulation. CTA/END reuse predictions across the sixteen image-view queries of one target. The quality and timing cohorts differ; these panels do not constitute a whole-planner Pareto benchmark.

for CTA-DIR at $L = 15$ gives $p = .05045$, so that comparison should not be described as an established significance result. No correction for multiple exploratory comparisons is applied. Mean-score contrast intervals cross zero. Ranking gains on fixed banks therefore do not establish a general closed-loop advantage over learned world-model alternatives.

5.4. Candidate and query scaling

Ranking with more candidates. A separate logged-bank experiment uses 2,810 P0 decisions from 100 development roots and scores banks with $K \in \{8, 16, 32, 64\}$. These are newly logged banks rather than exact prefixes of the earlier $K = 8$ run. Define mean geometry gain as $\Delta_K = |\mathcal{B}|^{-1} \sum_b (y_{b\hat{k}_b} - y_{b0})$. CTA’s gain rises from $.608 \times 10^{-3}$ at $K = 8$ to $.999 \times 10^{-3}$ at $K = 64$ (Fig. 4a); the saved paired increase is $.391 \times 10^{-3}$, with 95% root-bootstrap interval $[.328, .458] \times 10^{-3}$. Retained gap remains approximately .62. At $K = 64$, CTA-END gain is $.110 \times 10^{-3} [.064, .155] \times 10^{-3}$ and CTA-DIR-large is $.260 \times 10^{-3} [.172, .347] \times 10^{-3}$. This checks the ranking advantage under an expanded proposal workload, without treating it as a closed-loop or independent-test result.

Prediction and readout cost. Figure 4b,c measure the predictor/readout components for $K \in \{8, 16, 64\}$ and $G \in \{1, 16\}$. Measurements use synchronized CUDA execution on an H100 3g.40gb MIG instance, ten initial states, three warmups, and ten repeats per state. All three curves exclude the same shared context encoding, proposal sampling, and simulator execution. Repeated queries reuse the CTA and

END predictions; DIR evaluates its joint scorer again. The sixteen queries are image views of one PushT target pose, rather than sixteen distinct semantic goals.

At $K = 8$, CTA takes 4.11ms for one query and 16.87ms for sixteen, compared with END’s 4.52/23.40ms and DIR’s 2.54/26.81ms. Direct scoring is cheaper for one query at this candidate count. With sixteen queries, CTA reduces scorer time by 27.9% relative to END and 37.1% relative to DIR. At $K = 64$, CTA takes 12.96/114.42ms, versus END’s 20.09/171.04ms and DIR’s 13.94/206.67ms. The benefit depends on both the candidate batch and the repeated-readout workload.

The shared context path takes approximately 6.5ms, while the common diffusion proposal sampler takes 765ms at $K = 8$ and 923ms at $K = 64$. Proposal generation dominates this implementation, so component savings do not imply a comparable reduction in total control latency. DINO-WM’s separately measured encoding/rollout/cost path uses a different timing scope and precision; it is reported in the supplement rather than mixed into the matched component curves. End-to-end latency and peak memory remain to be benchmarked under a common precision and input protocol.

6. Discussion and Conclusion

CTA makes the future representation itself a learnable component of a visual world model. A context-conditioned encoder maps sampled action consequences to a finite segment code; the world model predicts that code, and a separate goal-conditioned reader evaluates the prediction. This provides a compact interface between action-conditioned dynamics and decisions while retaining visual future supervision during

target learning.

The present evidence supports two findings. The learned target improves geometry ranking on fixed development proposals, including over a parameter-matched direct scorer. Its parallel forecast and reusable readout also have a favorable component-cost profile in the measured repeated-query workload. Closed-loop development results show improvement over the proposal policy, while comparisons against learned alternatives remain uncertain. These findings motivate the representation without establishing a general planning or state-of-the-art advantage.

Limitations and open questions. The experiments cover one task, reused development roots, and one training seed. Historical proposal sampling is numerically sensitive to batch grouping; stronger control evidence requires independent roots and seeds under the corrected protocol. A large frozen encoder and pretrained proposal policy contribute substantially to deployment cost, and CTA cannot choose actions outside the proposal bank. The code is learned with privileged training labels, and the relationship between those labels and native task success is imperfect.

The current comparisons do not separately attribute the gain to conditioning, intermediate observations, quantization, or future supervision. A factorized direct comparator and a compact per-frame world model are especially relevant controls. The decoder encourages future information but does not by itself guarantee that the code retains all decision-relevant events. Forecast distributions are reduced to coordinate means, without a calibrated uncertainty treatment. Finally, multiple views of one target do not test new semantic goals. Such goals and tasks with consequential intermediate events would help characterize what the segment representation preserves.

CTA consequently offers a concrete direction for world-model design: forecast a learned description of an action segment in the context where it will be used. Establishing its broader impact requires the representation-quality and component-compute findings to carry through to reproducible closed-loop control across tasks and workloads.

References

- [1] Dilip Arumugam and Benjamin Van Roy. Deciding what to model: Value-equivalent sampling for reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 9024–9044, 2022. 3
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. 2
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2
- [4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 5
- [5] Hanchen Cui, Sergio Arnaud, Arjun Majumdar, Daniel Dugas, Elie Aljalbout, Karthik Desingh, Krishna Murthy Jatavallabhula, and Franziska Meier. DREAMSTEER: Latent world models can steer VLA policies during deployment without any finetuning. *arXiv preprint arXiv:2607.02865*, 2026. 2, 3
- [6] Maximilian Du and Shuran Song. DynaGuide: Steering diffusion policies with active dynamic guidance. In *Advances in Neural Information Processing Systems*, pages 44192–44221, 2025. 3
- [7] Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 5541–5552, 2020. 3
- [8] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2555–2565, 2019. 1, 2
- [9] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 3
- [10] Zhengyao Jiang, Tianjun Zhang, Michael Janner, Yueying Li, Tim Rocktäschel, Edward Grefenstette, and Yuandong Tian. Efficient planning in a compact latent action space. In *International Conference on Learning Representations*, 2023. 1, 3
- [11] Dongwon Kim, Gawon Seo, Jinsung Lee, Minsu Cho, and Suha Kwak. Planning in 8 tokens: A compact discrete tokenizer for latent world model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 1, 3
- [12] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Proceedings of the 9th Conference on Robot Learning*, pages 3200–3217, 2025. 3
- [13] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. In *Advances in Neural Information Processing Systems*, pages 18114–18125, 2021. 1, 3
- [14] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, pages 51772–51783, 2024. 3, 4

- 567 [15] Vincent Micheli, Eloi Alonso, and François Fleuret. Effi-
568 cient world models with context-aware tokenization. In *Pro-*
569 *ceedings of the 41st International Conference on Machine*
570 *Learning*, pages 35623–35638, 2024. 1, 3
- 571 [16] Mitsuhiro Nakamoto, Oier Mees, Aviral Kumar, and Sergey
572 Levine. Steering your generalists: Improving robotic foun-
573 dation models via value guidance. In *Proceedings of the 8th*
574 *Conference on Robot Learning*, pages 4996–5013, 2025. 1, 3
- 575 [17] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V.
576 Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez,
577 Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido
578 Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes,
579 Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rab-
580 bat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou,
581 Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bo-
582 janowski. DINOv2: Learning robust visual features without
583 supervision. *Transactions on Machine Learning Research*,
584 2024. 1, 3
- 585 [18] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang.
586 Inference-time enhancement of generative robot policies via
587 predictive world modeling. *IEEE Robotics and Automation*
588 *Letters*, 2025. Accepted for publication. 2, 3
- 589 [19] Sruthi Sudhakar, Junbang Liang, Sreehari Rammohan, Pavel
590 Tokmakov, Richard Zemel, and Carl Vondrick. Robot critics
591 that sweat the small stuff. *arXiv preprint arXiv:2606.21572*,
592 2026. 3
- 593 [20] Yilin Wu, Thomas Tian, Gokul Swamy, and Andrea Bajcsy.
594 From foresight to forethought: VLM-in-the-loop policy steer-
595 ing via latent alignment. In *Proceedings of Robotics: Science*
596 *and Systems*, 2025. 3
- 597 [21] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto.
598 DINO-WM: World models on pre-trained visual features en-
599 able zero-shot planning. In *Proceedings of the 42nd Interna-*
600 *tional Conference on Machine Learning*, pages 79115–79135,
601 2025. 1, 2, 5